



UNIVERSIDAD
NACIONAL
DE LA PLATA

Minería de Datos y Aprendizaje Automático

Clase 5

Balance de clases - Selección de características - Sistemas de recomendación

2026

Temario

1

Técnicas de balance de clases

2

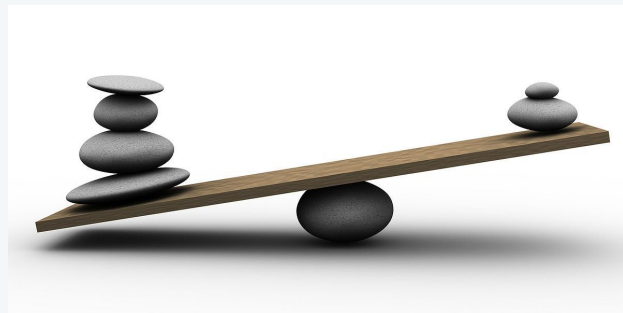
Selección de características

3

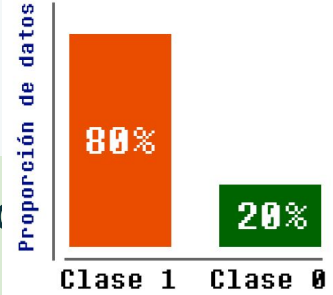
Sistemas de recomendación

01

Técnicas de balance de clases



Desbalance de clases



Se denomina “desbalance de clases” o “conjunto desbalanceado” a aquel dataset que presenta una diferencia entre el número de muestras pertenecientes a distintas clases.

En problemas de clasificación binaria:

- Se denomina “clase mayoritaria” a aquella clase que tiene la mayor proporción de las muestras.
- Se denomina “clase minoritaria” a aquella clase que posee la menor proporción de las muestras. Por lo general, es la clase más importante. Ej: Detección de enfermedades, riesgos crediticios, eventos climáticos, etc.

Desbalance de clases

El problema de trabajar con datasets desbalanceados es que los modelos terminan aprendiendo más de la clase mayoritaria, ya que intentan minimizar el error global.

Solución: balancear el dataset

- Undersampling
- Oversampling
- Ponderación de clases

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	0	20
	Negativo	0	25000

Submuestreo (undersampling)

La técnica de submuestreo consiste en balancear la distribución de los datos reduciendo el número de muestras de la clase mayoritaria y dejando intacta la cantidad de la clase minoritaria.

El problema de la técnica de submuestreo es que quedan muchas muestras fuera del dataset de entrenamiento.

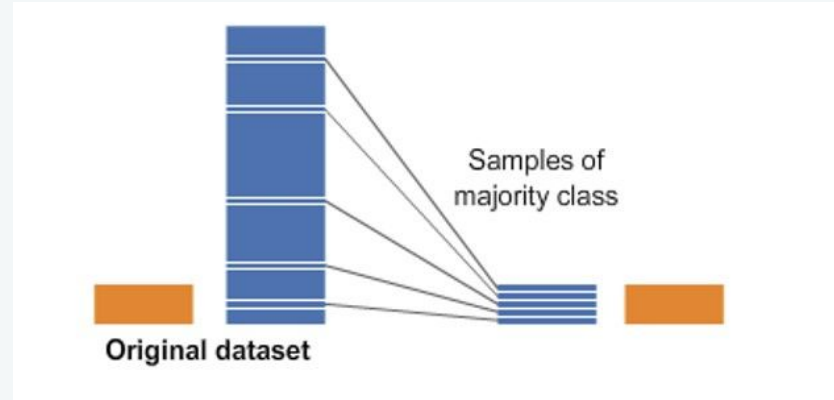
- Random Under Sampling
- Tomek Links
- NearMiss
- KNNs



Submuestreo (undersampling)

El submuestreo aleatorio (Random UnderSampling, RUS) consiste en eliminar muestras aleatorias de la clase mayoritaria.

La desventaja de este método es que, por azar, puede ignorar información potencialmente útil.

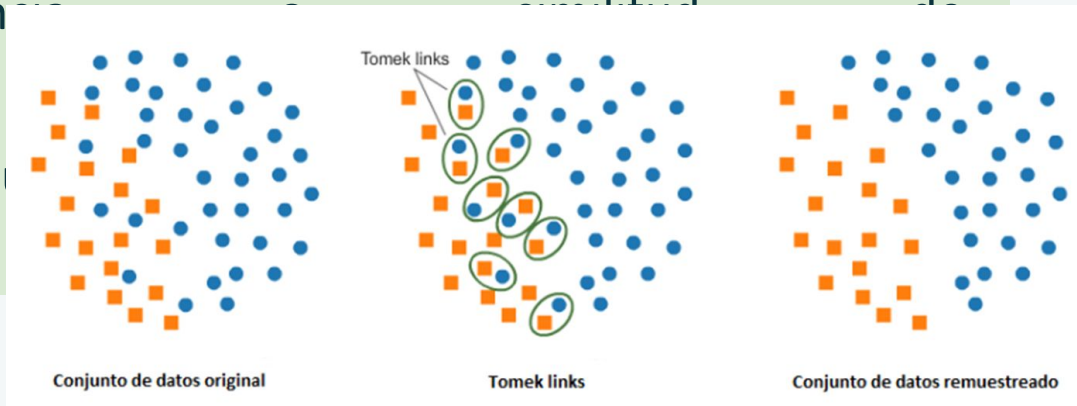


Submuestreo (undersampling)

La técnica “Tomek links” consiste en eliminar sólo muestras de la clase mayoritaria que sean redundantes o que estén muy próximas a muestras de la clase minoritaria.

Dos muestras x , y de diferentes clases son un “Tomek link” si para cualquier muestra z se cumple: $d(x,y) < d(x,z)$ y $d(x,y) < d(y,z)$, donde d es la distancia o similitud entre las muestras.

Podría introducirse un sesgo a favor de la clase minoritaria.



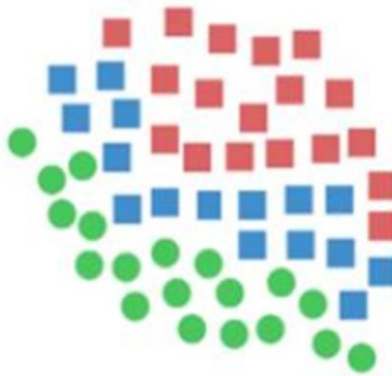
Submuestreo (undersampling)

La técnica “NearMiss” selecciona un número dado de muestras de la clase mayoritaria más próximas a cada muestra de la clase minoritaria.

Podría introducir sesgos en problemas multiclase y eliminar zonas “confiables” de alguna clase.



Conjunto de datos original



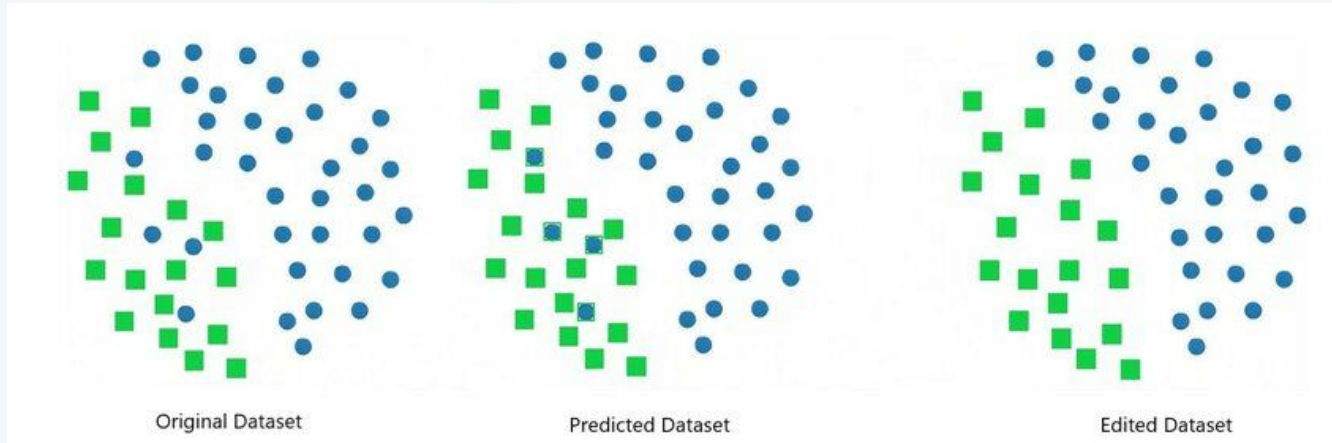
NearMiss



Conjunto de datos remuestreado

Edited nearest neighbours

Basado en el resultado de entrenar un algoritmo K-NN, esta técnica analiza los K vecinos más cercanos a cada muestra de la clase minoritaria, eliminando aquellas donde sus vecinos (la mayoría o todos según su configuración) pertenecen a clases distintas.



Undersampling

```
from imblearn.under_sampling import RandomUnderSampler
from imblearn.under_sampling import TomekLinks
from imblearn.under_sampling import NearMiss
from imblearn.under_sampling import EditedNearestNeighbours

rus = RandomUnderSampler(random_state=11)
X_resampled, T_resampled = rus.fit_resample(X, T)

tl = TomekLinks()
X_resampled, T_resampled = tl.fit_resample(X, T)

nms = NearMiss(version=1)
X_resampled, T_resampled = nms.fit_resample(X, T)

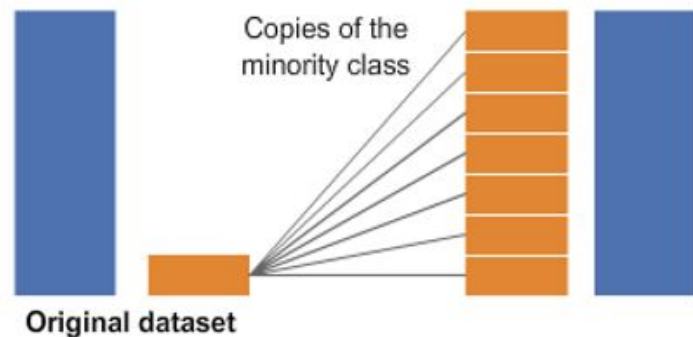
enn = EditedNearestNeighbours(kind_sel="all", n_neighbors=5)
X_resampled, T_resampled = enn.fit_resample(X, T)
# https://imbalanced-learn.org/stable/under\_sampling.html
```

Sobremuestreo (oversampling)

El sobremuestreo consiste en balancear la distribución de los datos incrementando el número de muestras de la clase minoritaria y dejando intacta la cantidad de la clase mayoritaria.

Añadir muestras a la clase minoritaria para balancear el conjunto de datos causa que el dataset de entrenamiento crezca en volumen.

- Random Over sampling
- SMOTE
- ADASYN

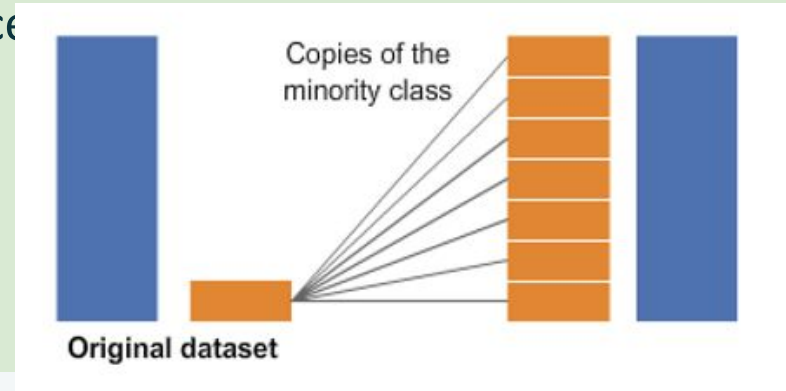


Sobremuestreo (oversampling)

El Sobremuestreo Aleatorio (Random OverSampling, ROS) consiste en equilibrar el conjunto de datos replicando las muestras de la clase minoritaria.

ROS aumenta la proporción de solapamiento entre instancias de diferentes clases, debido a la duplicación de datos, ya que la replicación de las muestras hace que la proporción de solapamiento entre los atributos sea mayor.

ROS es muy útil para conjuntos de datos con un índice de desequilibrio.

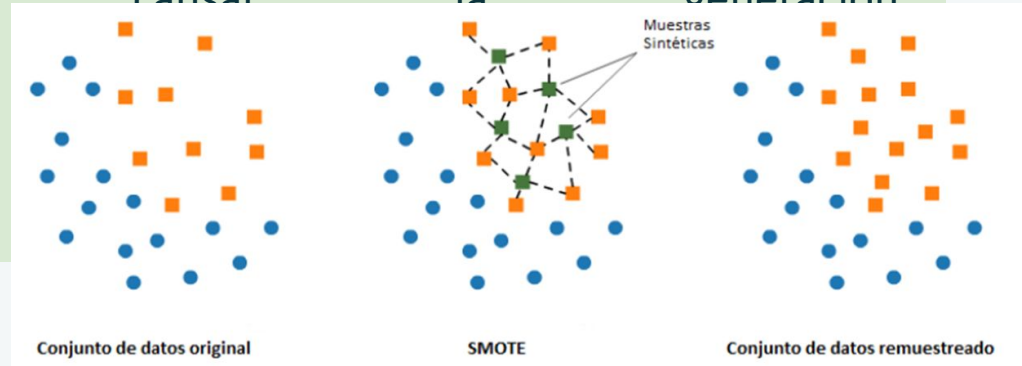


Sobremuestreo (oversampling)

El Sobremuestreo Sintético (Synthetic Minority Oversampling Technique, SMOTE) consiste en generar muestras sintéticas. SMOTE selecciona una muestra de la clase minoritaria y de entre sus vecinos más cercanos selecciona uno y crea una nueva muestra entre ambos.

Presenta un problema conocido como “generalización excesiva”, esto es, al generar nuevas muestras no tiene en cuenta las muestras vecinas de otras clases, lo que podría causar la generación de muestras “ruidosas”.

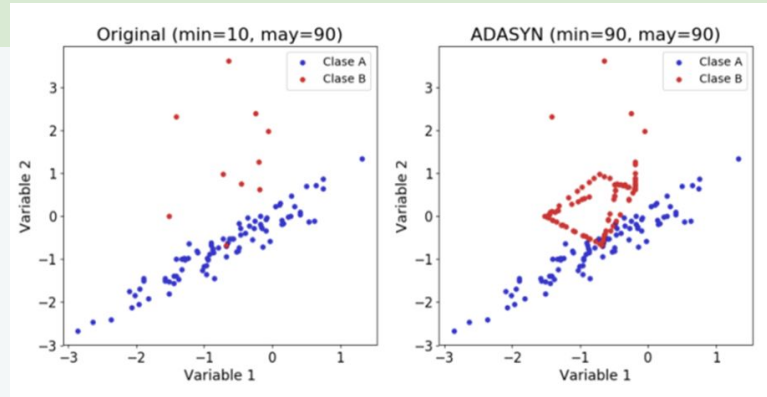
Existen muchas algoritmo.



Sobremuestreo (oversampling)

El Muestreo Sintético Adaptativo (Adaptive Synthetic Sampling Approach for Imbalanced Learning, ADASYN) es una extensión de SMOTE. ADASYN consiste en generar más muestras sintéticas en las zonas fronterizas para “aliviar” el problema del solape entre clases.

las nuevas muestras son generadas a partir de las muestras fronterizas, a través de una distribución de densidad. Esto causa que se vaya modificando el límite de decisión.



Oversampling

```
from imblearn.over_sampling import RandomOverSampler
from imblearn.over_sampling import SMOTE
from imblearn.over_sampling import ADASYN
```

```
ros = RandomOverSampler(random_state=0)
X_resampled, T_resampled = ros.fit_resample(X, T)
```

```
smt = SMOTE()
X_resampled, T_resampled = smt.fit_resample(X, T)
```

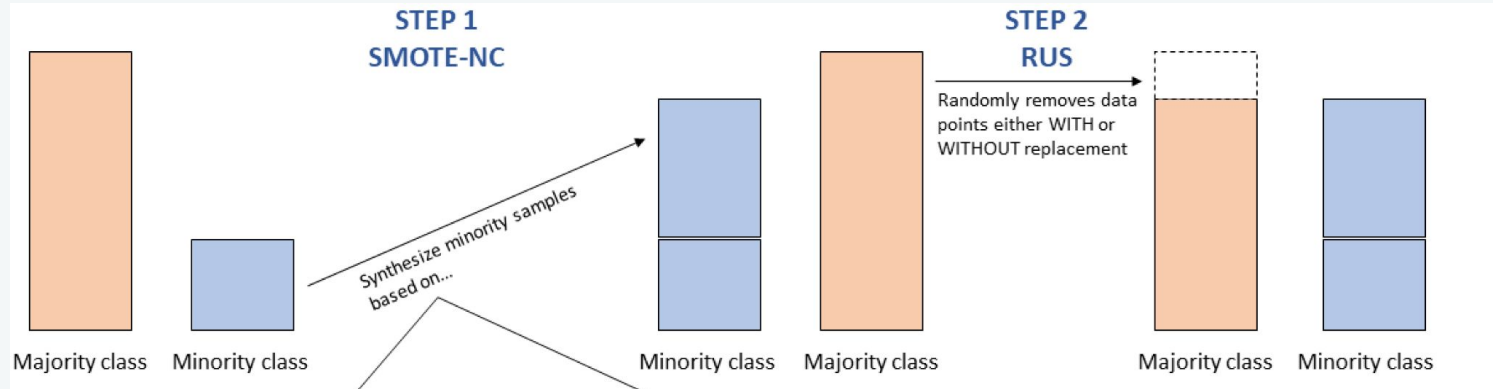
```
ada = ADASYN(sampling_strategy='minority')
X_resampled, T_resampled = ada.fit_resample(X, T)
```

```
# https://imbalanced-learn.org/stable/over\_sampling.html
```

Escenarios híbridos

Cuando el desbalance es muy marcado, se puede analizar la posibilidad de usar under y oversampling sobre el dataset de datos.

La clase minoritaria se aumenta mediante oversampling, al mismo tiempo que la clase mayoritaria se reduce utilizando subsampling.



Escenarios híbridos

```
from imblearn.combine import SMOTEENN
from imblearn.combine import SMOTETomek

smote_enn = SMOTEENN(random_state=17)
X_resampled, T_resampled = smote_enn.fit_resample(X, T)

smote_tomek = SMOTETomek(random_state=5)
X_resampled, T_resampled = smote_tomek.fit_resample(X, T)

# https://imbalanced-learn.org/stable/combine.html

# sampling_strategy: ratio para problemas binarios. Diccionario con
# cantidades de muestras para problemas multiclase.
over = RandomOverSampler(sampling_strategy=0.5)
under = RandomUnderSampler(sampling_strategy=0.8)
X_over, T_over = over.fit_resample(X, T)
X_resampled, T_resampled = under.fit_resample(X_over, T_over)
```

Ponderación de clases

Algunas técnicas penalizan de manera diferente los errores cometidos durante el entrenamiento.

Los errores de las muestras de la clase minoritaria tienen más peso que las muestras de la otra clase.

- Hiperparámetro “class_weight”.

Otras técnicas utilizan un desplazamiento del umbral de decisión. Aquellas que ofrecen un resultado probabilístico como respuesta, pueden desplazar el umbral a favor de la clase minoritaria.

Por ejemplo, al usar un umbral de 0.75, se establece que la respuesta a favor de la clase mayoritaria es tal, si la probabilidad es mayor a 0.75, caso contrario el algoritmo responde a favor de la clase minoritaria.

Ponderación de clases

```
from sklearn.ensemble import RandomForestClassifier

# Se debe definir en un diccionario, la ponderación de cada clase
cw = {0: 0.8, 1: 0.2}

rf1 = RandomForestClassifier(n_estimators = 100,
                             class_weight = cw)

rf2 = RandomForestClassifier(n_estimators = 100,
                             class_weight = "balanced")
# cada clase toma su peso de acuerdo a su
# proporción en el dataset

# Similar en otros modelos
```



Actividades

1. Utilice los diferentes métodos de under y over sampling vistos para mejorar la detección de la clase minoritaria en el dataset “glass”.

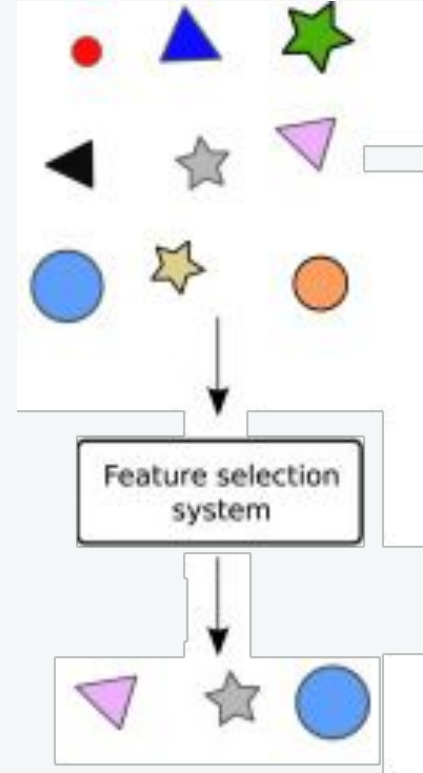
Utilice un método de ensamble para la tarea de clasificación.

Con el mismo ensamble entrene diferentes modelos usando para cada uno un dataset tratado con cada una de las técnicas de balance estudiadas.

Realice la comparación de los resultados obtenidos de accuracy, precision y recall (estas últimas dos, de la clase minoritaria).

02

Selección de características



Selección de características

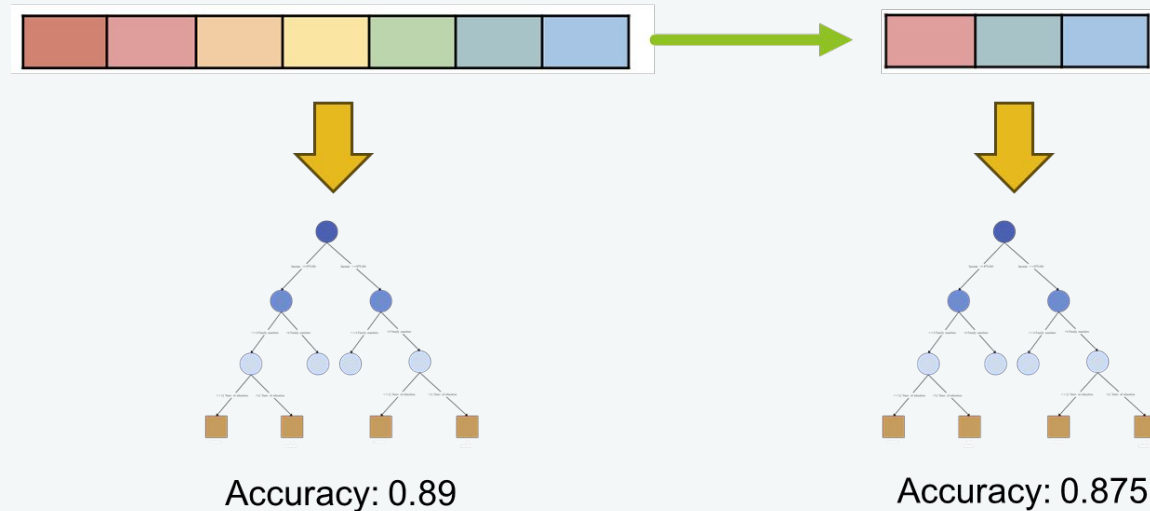
La selección de características (feature selection) es un área dentro del proceso de extracción de conocimiento. Las técnicas dentro de esta área tienen por objetivo identificar las variables más significativas para resolver una determinada tarea.

Ventajas

- Permite reducir la dimensionalidad del dataset.
- Mejora la interpretabilidad del modelo y los resultados.
- Reduce los costos de obtención y almacenamiento de la información.
- Mejora la eficiencia computacional de los algoritmos que hacen uso de la información.
- Ayuda a mitigar el sobreajuste en los algoritmos de machine learning.

Selección de características

El objetivo es encontrar un subconjunto de las variables, atributos o características (columnas) de un dataset, tal que con ese subconjunto se pueda conseguir un modelo de rendimiento similar al que se obtendría utilizando el dataset completo.



Selección de características

Dado un dataset de dimensión N existen $2^N - 1$ subconjuntos distintos.

Con 30 variables $\rightarrow$ 1.073.741.823 de combinaciones posibles!!!!

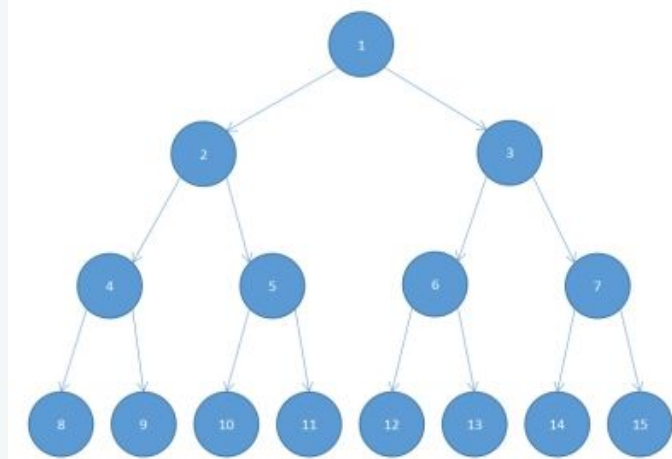
Existen varios enfoques que permiten realizar una selección de características.

- Búsqueda ciega
- Métodos de filtrado
- Métodos empaquetados (wrapper)
- Métodos integrados
- Meta-heurísticas

Búsqueda ciega (Blind search)

La búsqueda ciega consiste en la evaluación del conjunto de soluciones completo. Es decir, dado un conjunto de N características se evalúan las $2^N - 1$ posibles combinaciones. Se selecciona la mejor de todas.

Con 5 variables $\rightarrow$ 31 combinaciones



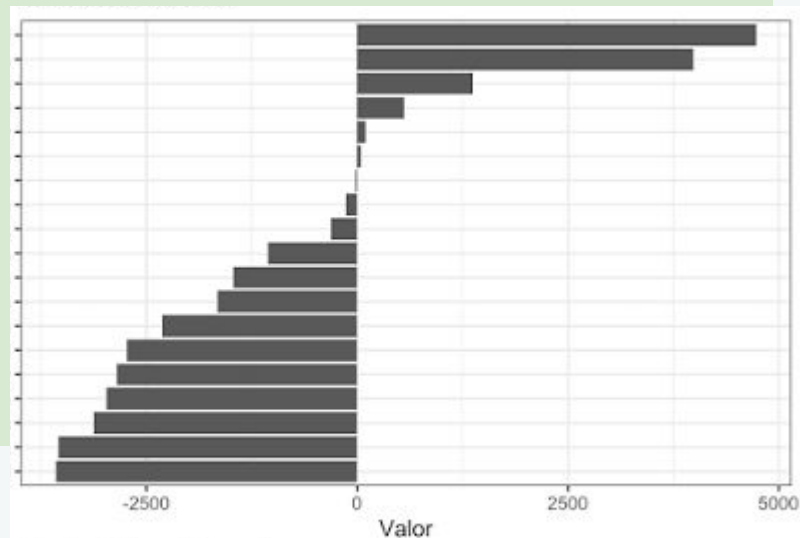
Métodos de filtrado (Filter Methods)

Evalúan la relevancia de cada característica de forma individual sin ningún tipo de sinergia ni combinación entre características.

La relevancia calculada tiende a determinar la correlación de la característica con la variable a predecir.

Las características se
con algún índice de
de seleccionar aquellas
cierto valor.

Es una técnica
características rápida
del modelo.



Métodos de filtrado (Filter Methods)

Métodos de filtrado más comunes:

- Varianza
- Coeficiente de correlación de Pearson
- Test Chi-cuadrado
- Información mutua
- F-test ANOVA
- Entropía
- Gini
- ...

$$\text{Variance}(X) = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

$$MI(X, Y) = \sum_{x,y} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right)$$

$$F = \frac{\text{Variance Between Groups}}{\text{Variance Within Groups}}$$

Métodos de filtrado

```
from sklearn.feature_selection import VarianceThreshold

variance_selector = VarianceThreshold(threshold=0.1)
X_transform = variance_selector.fit_transform(X)

# La función "corrwith" es de Pandas
correlation_scores = X.corrwith(T)
top_features = correlation_scores.sort_values(ascending=False)

print(top_features.index.tolist())
```

Métodos de filtrado

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
from sklearn.feature_selection import mutual_info_classif
from sklearn.feature_selection import f_classif

chi_selector = SelectKBest(score_func=chi2, k=10)
X_transform = chi_selector.fit_transform(X, T)
print(chi_selector.get_feature_names_out()) # Características elegidas

mi_selector = SelectKBest(score_func=mutual_info_classif, k=10)
X_transform = mi_selector.fit_transform(X, T)
print(mi_selector.get_feature_names_out())

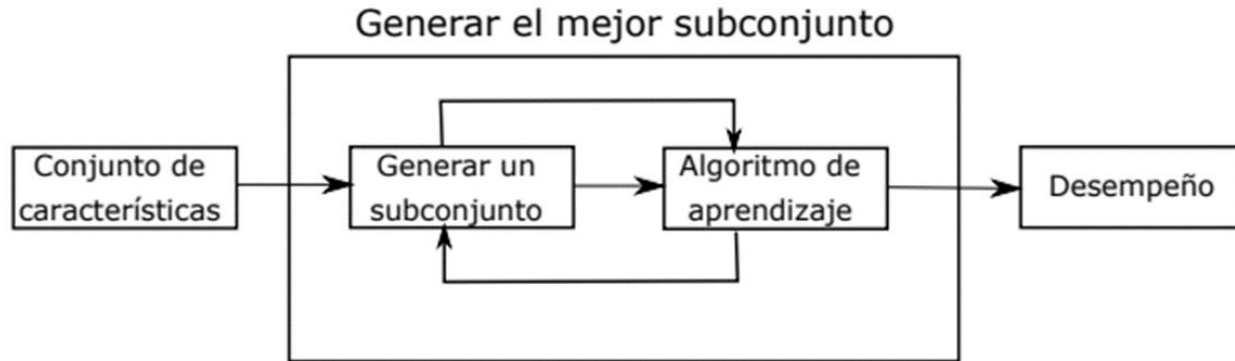
anova_selector = SelectKBest(score_func=f_classif, k=10)
X_transform = anova_selector.fit_transform(X, T)
print(anova_selector.get_feature_names_out())
```

Métodos empaquetados (Wrapper methods)

Son métodos iterativos.

Utiliza un subconjunto de características y se entrena un modelo con el subconjunto seleccionado.

A partir de los resultados obtenidos se agregan o eliminan características del subconjunto.



Métodos empaquetados (Wrapper methods)

Estrategias:

1. Forward selection.
Comienza con el conjunto vacío, va agregando características, una por una, hasta que la performance del modelo no cambie significativamente.
2. Backward elimination
Comienza con un conjunto formado por todas las características, mide el modelo, y luego va sacando características mientras la performance no decaiga significativamente.
3. Recursive feature elimination (RFE)
Parecido al anterior, solo que arma un ranking inicial de variables y comienza a eliminar desde la característica con peor ranking.

Métodos de empaquetado

```
from sklearn.feature_selection import RFE

model_base = RandomForestClassifier()

rfe = RFE(estimator=model_base, n_features_to_select=5)
ranking = rfe.fit(X, T)

print(ranking.get_feature_names_out())
```

Métodos integrados (Embedded methods)

Combinan partes de los métodos de filtrado y de empaquetado. Realizan la selección de las características durante el entrenamiento del modelo.

Son más rápidos que los métodos de empaquetado y a menudo más eficaces que los métodos de filtrado.

Por lo general se implementan como parte del algoritmo de entrenamiento.

Los árboles de decisión los utilizan para elegir la mejor característica al momento de dividir un nodo.



Métodos integrados (Embedded methods)

Regresión LASSO (Least Absolute Shrinkage and Selection Operator), también llamada L1. Es una regresión lineal que intenta hacer que todos sus coeficientes tiendan a cero.

$$\text{Loss} = \text{MSE} + \lambda \sum_{j=1}^n |w_j|$$

Regresión Ridge (L2). Similar a la anterior, penalizando con más fuerza los coeficientes más grandes.

$$\text{Loss} = \text{MSE} + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$$

Regresión logística regulada. Es como L1, pero realizando una regresión logística para tareas de clasificación.

$$\text{Loss} = -\log\text{-likelihood} + \lambda \sum |w_j|$$

L1-SVM. Máquinas de soporte vectorial con regularización L1.



Métodos integrados (Embedded methods)

```
from sklearn.linear_model import Lasso
```

```
las = Lasso(alpha=0.1)
```

```
model = las.fit(X_train, T_train)
```

Meta heurísticas

Una metaheurística es un algoritmo de alto nivel diseñado para encontrar soluciones buenas o aproximadas a problemas complejos de optimización.

Ejemplos:

- Simples
 - Recocido simulado (simulated annealing)
 - Tabú search
- Poblacionales
 - Algoritmos genéticos
 - Optimización por cúmulo de partículas (PSO, Particle Swarm Optimization)
 - Algoritmo de agujero negro (BHA, Black Hole Algorithm)

Meta heurísticas

Las metaheurísticas trabajan sobre un espacio en R , N dimensional, de posibles soluciones. N = número de características. Cada dimensión (por lo general representada entre 0 y 1, corresponde a una característica.

Evalúan un modelo de clasificación cientos o miles de veces.

Ejemplo.

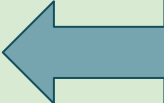
Una posible solución encontrada por el algoritmo

0.22	0.14	0.78	0.91	0.02	0.41	0.78
------	------	------	------	------	------	------

Redondeando

0	0	1	1	0	0	1
---	---	---	---	---	---	---

El resultado encontrado podría no ser el óptimo.



Las características con "1" son las elegidas.

Meta heurísticas

Pseudoalgoritmo de una meta heurística poblacional:

1. Inicializar cada individuo (posible solución) de la población (generación 0)
2. Evaluar los individuos (con una función de aptitud previamente establecida)
3. Iterar hasta converger o hasta alcanzar un máximo de iteraciones
 - a. Intercambiar información entre individuos
 - b. Obtener la nueva generación
 - c. Evaluar los individuos de la nueva generación

El mejor individuo (el que tiene el mejor score según la función de aptitud establecida) es la solución al problema.

Al ser un algoritmo poblacional, se podrían estudiar las soluciones representadas por varios individuos.

Actividades

2. Utilice los métodos de filtrado y empaquetado estudiados para reducir las características que usará un modelo de clasificación que será entrenado con la base de datos “glass aumentada” (alguna resultante de la actividad 1, luego de aplicar alguna técnica de balance de clases).

Como modelo de clasificación utilice algún modelo de ensemble.

Determine el valor de accuracy utilizando todos los atributos del dataset. Compare este valor con los obtenidos por los diferentes modelos que se entrenan sobre el dataset reducido (luego de aplicar la selección de características).

Utilice la técnica de cross-validation para obtener el accuracy de cada modelo.

03

Sistemas de recomendación



Sistemas de recomendación

Un algoritmo de recomendación es un sistema de toma de decisiones o de puntuación diseñado para sugerir elementos, contenido o acciones relevantes a los usuarios en función de sus preferencias, comportamientos y datos contextuales.

- Filtrado basado en contenido: Sugiere elementos similares a los ya consumidos, basándose en las características del propio contenido
- Filtrado colaborativo: Recomienda elementos analizando los patrones de usuarios con gustos parecidos (el principio de "usuarios que llevaron esto también les gustó aquello").

Pueden basarse en reglas, en similitudes o utilizar aprendizaje automático.

El objetivo principal de los algoritmos de recomendación es mejorar la experiencia del usuario y la toma de decisiones al proporcionar sugerencias personalizadas que reduzcan la sobrecarga de información.

Recomendación basada en contenido

Los filtros basados en contenido tienen el ítem como base de la predicción, en lugar de tener al usuario. Es decir, utiliza las características del ítem (marca, precio, calificaciones, tamaño, categoría, etc.) para hacer las recomendaciones. Aunque también se puede hacer la recomendación inversa (a qué usuarios les interesa un ítem).

Estos sistemas poseen información de ítems y usuarios.

- El perfil del ítem (o artículo) es la representación del ítem. Consiste en el conjunto de características de un elemento, que pueden ser características internas o metadatos descriptivos.
- El perfil del usuario representa las preferencias y el comportamiento del usuario. Puede consistir en representaciones de aquellos elementos en los que un usuario mostró interés previamente. También consta de datos de usuario de las interacciones pasadas con el sistema (por ejemplo, preferencias, calificaciones, consultas, etc.).

Recomendación basada en contenido

Para el armado de los perfiles se arma una matriz, donde las filas son los ítem y las columnas son los atributos que se pueden medir de cada ítem.

Cada fila (ítem) es un vector en un espacio N-dimensional. Los vectores representantes de ítems similares estarán cerca en el espacio N-dimensional.

	Attr. 1	Attr. 2	Attr. 3	Attr. 4	Attr. 5	Attr. 6	Attr. 7	...	Attr. M
Ítem 1	1	0	1	0	0	0	1	...	0
Ítem 2	0	1	1	0	0	0	0	...	0
Ítem 3	0	0	0	0	1	0	0	...	1
Ítem 4	0	0	0	1	0	1	0	...	1
Ítem 5	0	0	0	0	1	0	0	...	1
...	...	...	...	...	...	...	...	...	...
Ítem N	1	0	1	0	0	1	0	...	0

Sabiendo que ítems les interesa a un usuario, solo hay que buscar los similares (nuevos) y realizar esa recomendación.

Recomendación basada en contenido

Para determinar cuán similares son dos ítems hay que seleccionar un mecanismo de comparación (hay muchas opciones).

Una posibilidad es usar la distancia o similitud coseno, que mide el coseno del ángulo que forman dos vectores.

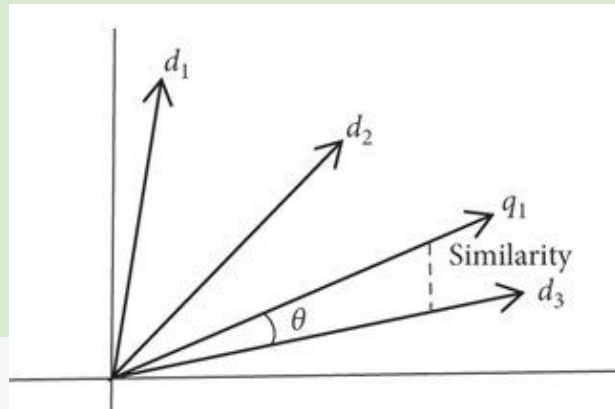
$$\cos\theta = \frac{\sum_0^{n-1} (a_i \cdot b_i)}{\sqrt{\sum_0^{n-1} a_i^2} \cdot \sqrt{\sum_0^{n-1} b_i^2}}$$

=1, los vectores apuntan exactamente en la misma dirección (ángulo de 0°).

=0, los vectores son ortogonales (ángulo de 90°).

=-1, los vectores apuntan en direcciones completamente opuestas.

Con lo cual, dado un ítem, solo resta buscar los ítems más parecidos (según la función de similitud usada), y esos serán los ítems a recomendar.



Recomendación basada en contenido

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

item = [[0,1,1,0,0,0,0,1]] # Una muestra ejemplo, se desea buscar los
                             # 5 ítems más parecidos

c_sim = cosine_similarity(X, item) # Obtiene la similitud de la
                                    # muestra contra todo el dataset

index_ord = np.argsort(c_sim, axis=0)
top5 = index_ord[-5:].squeeze() # Obtiene los índices del dataset
                                # donde están los 5 ítems más
                                # similares

print(c_sim[top5]) # Imprime los valores de similitud
print(X.iloc[top5]) # Imprime los 5 ítems más similares
```

Filtrado colaborativo

El filtrado colaborativo es una técnica que predice los intereses de un usuario basándose en las valoraciones y el comportamiento de otros usuarios con gustos similares.

Trabajan con una matriz de interacción que relaciona a los usuarios con los ítems (artículos) que han calificado. Los valores de la matriz es la puntuación que le dio el usuario X al ítem Y . Suele estar “casi” vacía (matriz sparse).

	Ítem 1	Ítem 2	Ítem 3	Ítem 4	Ítem 5	Ítem 6	Ítem 7	...	Ítem M
User 1	5		4				5	...	
User 2		1	2	0				...	
User 3				0	1			...	4
User 4				3		1		...	5
User 5	2	3			1		4	...	1
...	...	...	...	...	...	...	...	...	...
User N	1		5			2		...	

Filtrado colaborativo

El objetivo es “rellenar” los espacios en blanco.

Los valores “recomendados” estarán dentro de la escala del puntaje, luego habrá que seleccionar los de mayor puntaje para realizar la recomendación.

Existen muchas opciones para realizar esta tarea. Una es la conocida como “filtrado colaborativo basado en usuarios con normalización de la media”.

El cálculo del valor de recomendación del ítem i para el usuario u .

	Ítem 1	Ítem 2	Ítem 3	Ítem 4	...	Ítem M
User 1	5		4		...	
User 2		1	2	0	...	
User 3				0	...	4
User 4				3	...	5
User 5	2	3			...	1
...	...	...	...	...	...	...
User N	1		5		...	

Se calculan las similitudes del usuario u contra todos los que puntuaron el ítem i .

Similitud
1.72
3.01
4.32

Filtrado colaborativo

Cálculo del valor de recomendación del ítem i para el usuario u .

	Ítem 1	Ítem 2	Ítem 3	Ítem 4	...	Ítem M	Similitud
User 1	5		4		...		1.72
User 2		1	2	0	...		...
User 3	1.35	3.56	4.12	0	...	4	...
User 4				3	...	5	...
User 5	2	3			...	1	3.01
...	...	...	...	...	...	...	...
User N	1		5		...		4.32

$$V(u, i) = R_u + \frac{\sum_o Sim(u, o) \cdot (R_{o,i} - R_v)}{\sum_o Sim(u, o)}$$

$Sim(u, o)$ es la similitud entre los usuarios u y o .
 R_u es el promedio de los valores puntuados.
 $R_{o,i}$ es el ranking del usuario o al ítem i .

La principal desventaja de esta técnica es que no resulta muy exacta para usuarios nuevos, o que tienen pocas puntuaciones (problema del arranque en frío).

Filtrado colaborativo

```
# X_nan es un dataframe con "nan" para los ítems no puntuados.
usuario_id = 12      # El usuario a quien recomendar ítems
item_i = 108         # El ítem a calcular su score
n_cols = X_nan.shape[1]  # El número de atributos del dataset

Ru = X_nan.iloc[usuario_id].mean()

numerador = 0.0; denominador = 0.0
for usuario_v in X_nan.index:  # Iterar sobre todos los usuarios
    if usuario_v == usuario_id:
        continue
    if not np.isnan(puntua):  # Si tiene una calificación válida
        # Sumar la distancia entre el usuario_id y usuario_v (a continuación...)

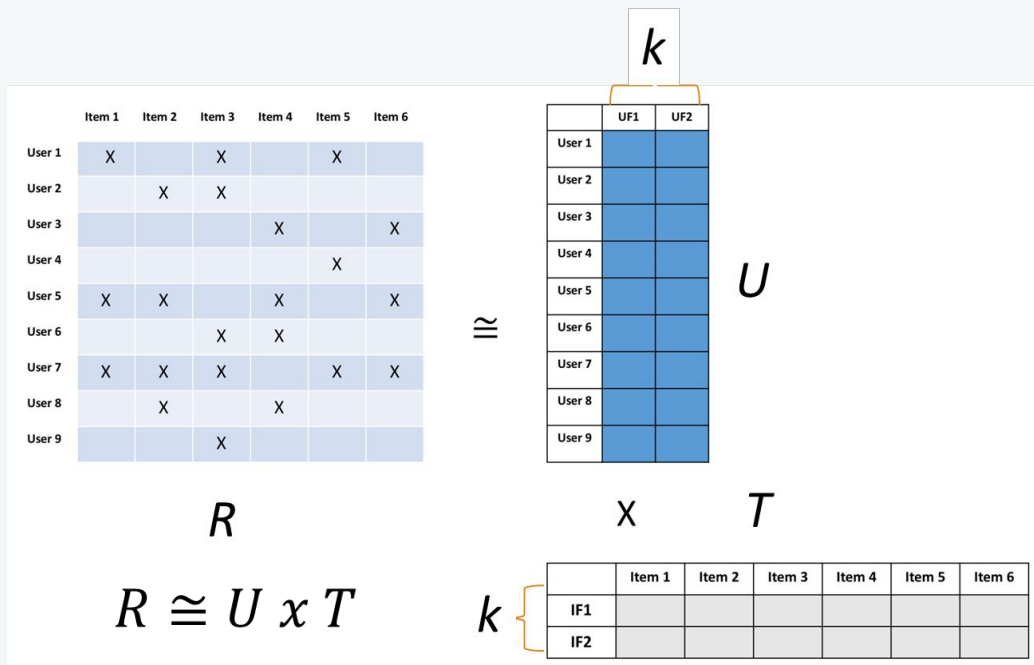
# El score para (usuario_id, item_i)
if denominador == 0:
    score = Ru
else:
    score = Ru + (numerador / denominador)
```

Filtrado colaborativo

```
# if ...  
    # Sumar la distancia entre el usuario_id y usuario_v  
    sim_u_v_num = 0; sim_u_v_den_a = 0; sim_u_v_den_b = 0  
    for i in range(n_cols):  
        a = X_nan.iloc[usuario_id, i]  
        b = X_nan.iloc[usuario_v, i]  
        if not np.isnan(a) and not np.isnan(b):  
            sim_u_v_num+= a * b  
            sim_u_v_den_a+= a * a  
            sim_u_v_den_b+= b * b  
    den = (np.sqrt(sim_u_v_den_a) * np.sqrt(sim_u_v_den_b))  
    if den != 0:  
        sim_u_v = sim_u_v_num / den  
        Rv = X_nan.iloc[usuario_v].mean()  
  
        numerador += sim_u_v * (puntuacion - Rv)  
        denominador += abs(sim_u_v)  
  
# if ...
```

ALS (Alternating least square)

El algoritmo ALS (Alternating least square) intenta descubrir dos matrices (T y U), tal que el producto de ellas sea la matriz de recomendación R.

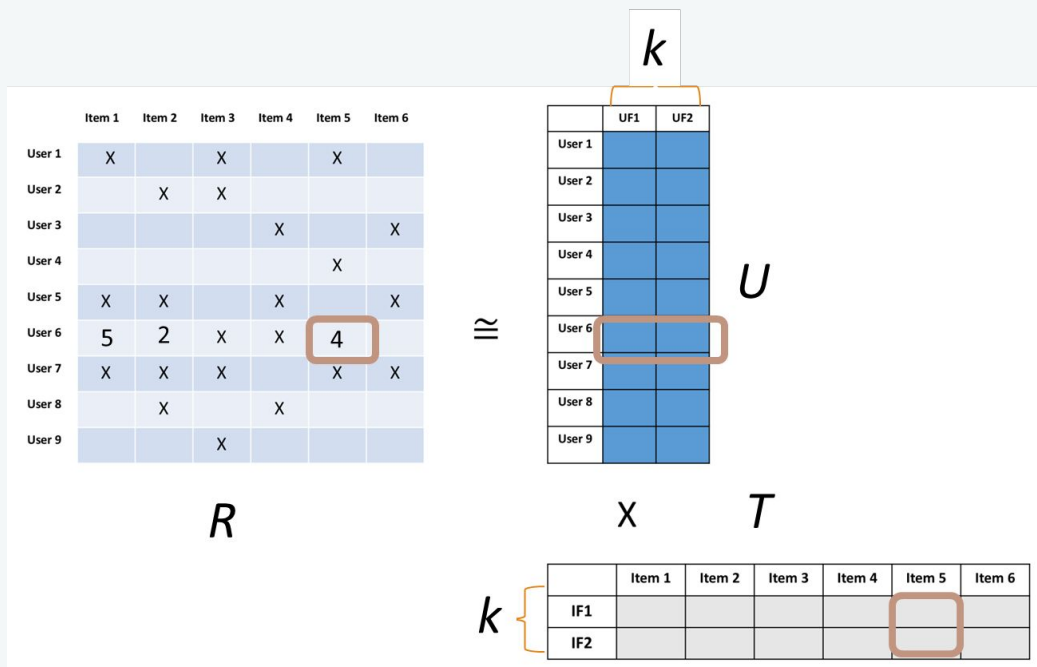


$$U_j = \left[\sum_{i; r_{ji} > 0} (T_i T_i^t) + \lambda I_k \right]^{-1} \sum_{i; r_{ji} > 0} (R_{ji} T_i)$$

$$T_j = \left[\sum_{i; r_{ji} > 0} (U_i U_i^t) + \lambda I_k \right]^{-1} \sum_{i; r_{ji} > 0} (R_{ji} U_i)$$

ALS (Alternating least square)

Las recomendaciones salen de multiplicar la fila de la matriz U correspondiente al usuario, con la columna de la matriz T para un determinado ítem.

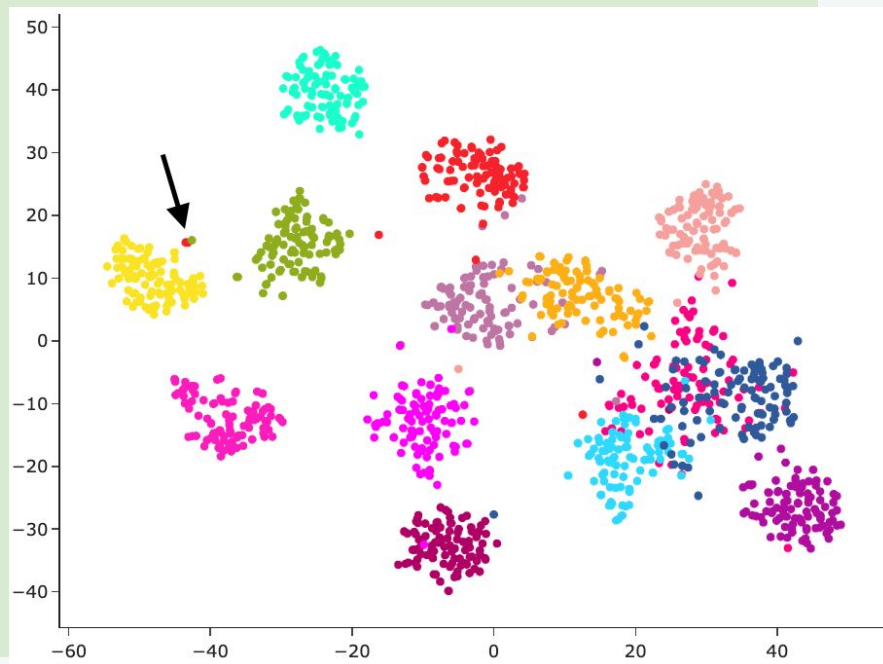


Filtrado colaborativo utilizando embeddings

Alternativas más modernas utilizan el concepto de vector de embeddings. A cada usuario y a cada ítem se le calcula su vector e embedding usando modelos más complejos (deep learning).

Los vectores de usuarios con gustos similares, o ítems con características similares, serán cercanos en el espacio de los vectores de embeddings (dimensionalidad alta: 128, 256, 512, o más).

Luego, utilizando alguna métrica de similitud, se pueden realizar las recomendaciones buscando los ítems de usuarios similares, o de ítems con características similares a las consumidas por un usuario.



Actividades

El dataset “movies” tiene el nombre de casi 4000 películas y los géneros a los cuales pertenece cada una.

El formato del archivo es:

movie_id::título y año::generos

Los géneros están separados por “|”

3. Use la base de datos “movies” para realizar recomendaciones basadas en contenido usando la similitud coseno.

Elija una película entre las 3883 que tiene el dataset y devuelva el top 10 de las películas a recomendar.

Use:

```
X_matrix = X['generos'].str.get_dummies(sep='|')
```

Para convertir el dataset “movies” en la matriz que se necesita.

Actividades

El dataset “ratings” tiene la puntuación (de 1 a 5) otorgada a películas por los usuarios que las vieron.

El formato del archivo es:

user_id::movie_id::puntuación::timestamp

4. Use la base de datos “ratings” para realizar recomendaciones por filtrado colaborativo basado en usuarios con normalización de la media.

Elija un usuario entre los 6040 que tiene el dataset y calcule la recomendación para cada película no vista.

Use:

```
X_matrix = pd.pivot_table(X, values='puntuación', index='user_id',  
                           columns='movie_id').fillna(np.nan)
```

Para convertir el dataset “ratings” en la matriz que se necesita.

RESUMEN

- Técnicas de balance de clases
- Selección de características
- Sistemas de recomendación

